

1. Interactive Terminal E-Commerce Cart & Inventory Tracker

Scenario

You are building an interactive terminal interface for a backend inventory system. The program must repeatedly prompt an store administrator to enter item purchases or stock updates until they decide to check out or exit.

The script needs to process inputs dynamically, validate user commands, apply bulk discounts using conditional logic, and safely exit when requested.

Given Starter Data

Provide your students with an initial store catalog dictionary:

Python

Available inventory: Item Name -> Unit Price

```
catalog = { "laptop": 800, "mouse": 20, "keyboard": 50 "monitor": 150 }
```

Task Requirements

Write a Python script using a while True loop, input() functions, and if / elif / else statements to complete the following:

- a) Interactive Command Loop (while True):
 - a. Keep asking the user to type an item name to add to their order, or type checkout to finish, or type exit to cancel.
 - b. Example prompt: `input("Enter item to buy (or 'checkout' / 'exit'): ").lower()`
- b) Validation & Processing (if / elif / else):
 - a. If item exists in catalog: Add the price to a running total variable (grand_total) and print: `--> Added <Item> ($<Price>) to order.`
 - b. If item does NOT exist: Print an error message: `--> [ERROR] Item not found in catalog. Try again.`
 - c. If user types 'exit': Break out of the loop immediately without printing a final bill.
- c) Checkout & Conditional Discounts:
 - a. If user types 'checkout': Exit the loop and apply conditional discounts based on the grand_total:
 - i. If total is \$500 or more: Apply a 10% discount.
 - ii. If total is between \$200 and \$499: Apply a 5% discount.
 - iii. If total is under \$200: No discount.
 - b. Final Bill Output:
 - i. Print a clean receipt showing total price before discount, total saved, and final bill amount:

CHECKOUT RECEIPT

=====

Subtotal: \$870.0

Discount: \$87.0

Final Total: \$783.0

=====

2. Student Grade Evaluator & Class Performance Tracker

Scenario

You are tasked with building a batch processing module for a school management system. The platform needs an administrator to specify how many student records they want to process, gather each student's details through interactive terminal inputs, store the records in a structured dictionary, and then evaluate each student's status automatically.

Task Requirements

Write a Python script using **input()**, **int()** conversion, a **range()** loop, **dictionary manipulation**, and **conditional checks** to complete the following:

- a. **Initial Setup & Batch Count:**
 - i. Prompt the administrator for the total number of students to enter: `count = int(input("How many student entries do you want to create? "))`
 - ii. Create an empty dictionary named `student_records = {}`.
- b. **Dynamic Entry Loop (for i in range(count)):**
 - i. Loop exactly count times.
 - ii. For each student, ask for:
 1. Student Name (String)
 2. Score between 0 and 100 (Convert to float or int)
 - iii. Store the student in the dictionary using the student's name as the **key** and their score as the **value**: `student_records[name] = score`
- c. **Evaluation & Performance Analysis:**
 - i. Loop through `student_records.items()` to evaluate each student's score using conditional statements:
 1. **Score >= 70:** Grade **A** (Status: "Passed with Distinction")
 2. **Score >= 50:** Grade **B** (Status: "Passed")
 3. **Score < 50:** Grade **F** (Status: "Needs Improvement")
 - ii. Print each student's summary line during evaluation.
- d. **Class Summary:**
 - i. Calculate and print the **class average score** (sum of scores / count) and display the total number of students who passed versus failed.

Example Terminal Output

Plaintext

How many student entries do you want to create? 3

--- Entry 1 ---

Enter student name: Sarah

Enter score (0-100): 85

--- Entry 2 ---

Enter student name: David

Enter score (0-100): 42

--- Entry 3 ---

Enter student name: Alex

Enter score (0-100): 68

=====

EVALUATION RESULTS

=====

- Sarah: Score 85.0 | Grade A | Passed with Distinction

- David: Score 42.0 | Grade F | Needs Improvement

- Alex: Score 68.0 | Grade B | Passed

=====

CLASS PERFORMANCE

=====

Average Score: 65.0

Total Passed: 2

Total Failed: 1

3. Backend Data Processing & User Audit Tool

Scenario

You are building an automated administrative script for a web application backend. The platform needs to audit a raw list of incoming user sign-ups, check their permission levels, validate whether their accounts are active, and categorize them into different access groups for reporting.

Given Starter Data

Copy this starter list into your script: Python

Raw user records: (User ID, Name, Role, Is_Active, Login_Attempts)

```
users = [ (101, "Alice", "admin", True, 1),  
(102, "Bob", "member", True, 4),  
(103, "Charlie", "editor", False, 0),  
(104, "Diana", "admin", False, 6),  
(105, "Evan", "member", True, 2),  
(106, "Fiona", "guest", True, 0), ]
```

Task Requirements

Write a Python script using **for loops**, **if / elif / else statements**, and **while loops** (where applicable) to achieve the following:

- a. **Active Admin Processing (if / elif / else inside for loop):**
 - i. Loop through the users list.
 - ii. If a user is **active** (True) AND has the role **"admin"**, print: "[GRANT] Full system access granted to <Name> (ID: <ID>)"
 - iii. If a user is **active** (True) AND has the role **"member"** or **"editor"**, print: "[GRANT] Standard access granted to <Name> (ID: <ID>)"
 - iv. If a user is **inactive** (False), print: "[DENIED] Account <Name> is inactive."
- b. **Security Audit (while or conditional checks):**
 - i. Flag any account where Login_Attempts is **5 or greater** as a potential security risk: "[ALERT] Account <Name> is LOCKED due to excessive failed logins (<Attempts> attempts)."
- c. **Summary Counts:**
 - i. Maintain running counter variables during the loop to track total **active users**, total **inactive users**, and total **flagged accounts**.
 - ii. At the end of the script, display a clean summary report:

AUDIT SUMMARY REPORT

=====

Total Active Users Granted: X

Total Inactive Accounts: Y

Total Security Alerts: Z